Tasks on k8s-01

1.Setup Minikube in your local machine.

Install minikube on windows:

==========

1) Install Docker on windows.

https://docs.docker.com/desktop/install/windows-install/

2) Install Oracle box on windows

https://adamtheautomator.com/install-virtualbox-on-windows-10/

3) Check system info in CMD

systeminfo

4) Install Minikube using powershell

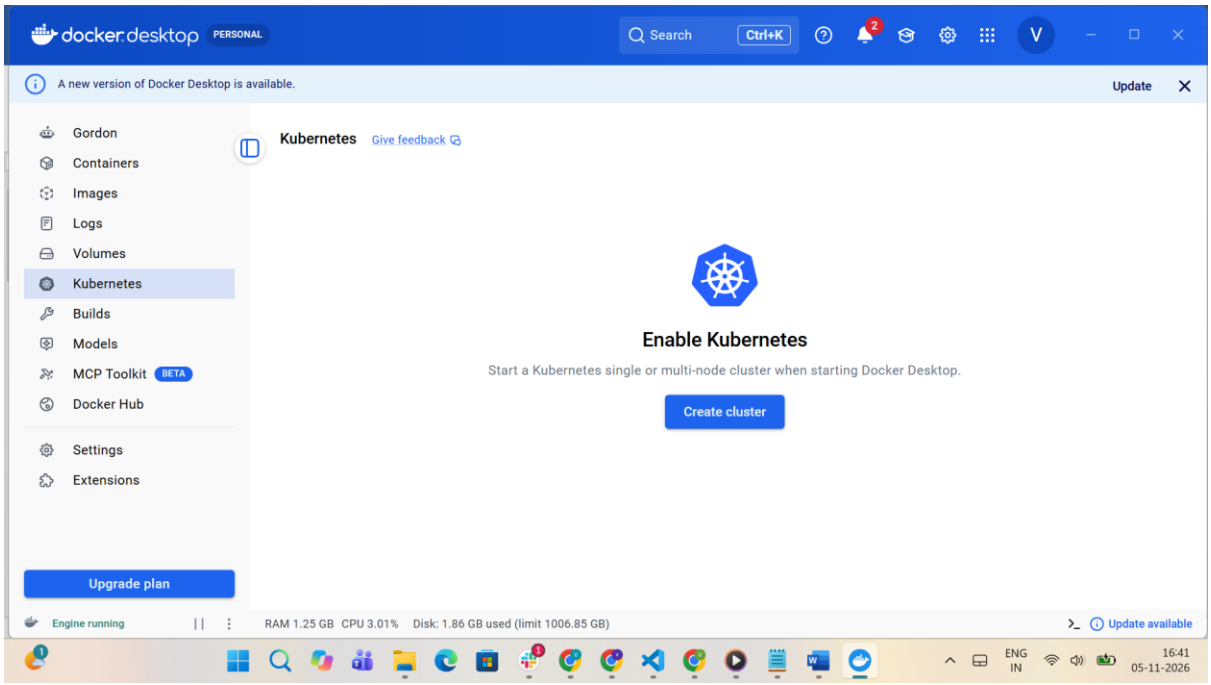
https://minikube.sigs.k8s.io/docs/start/

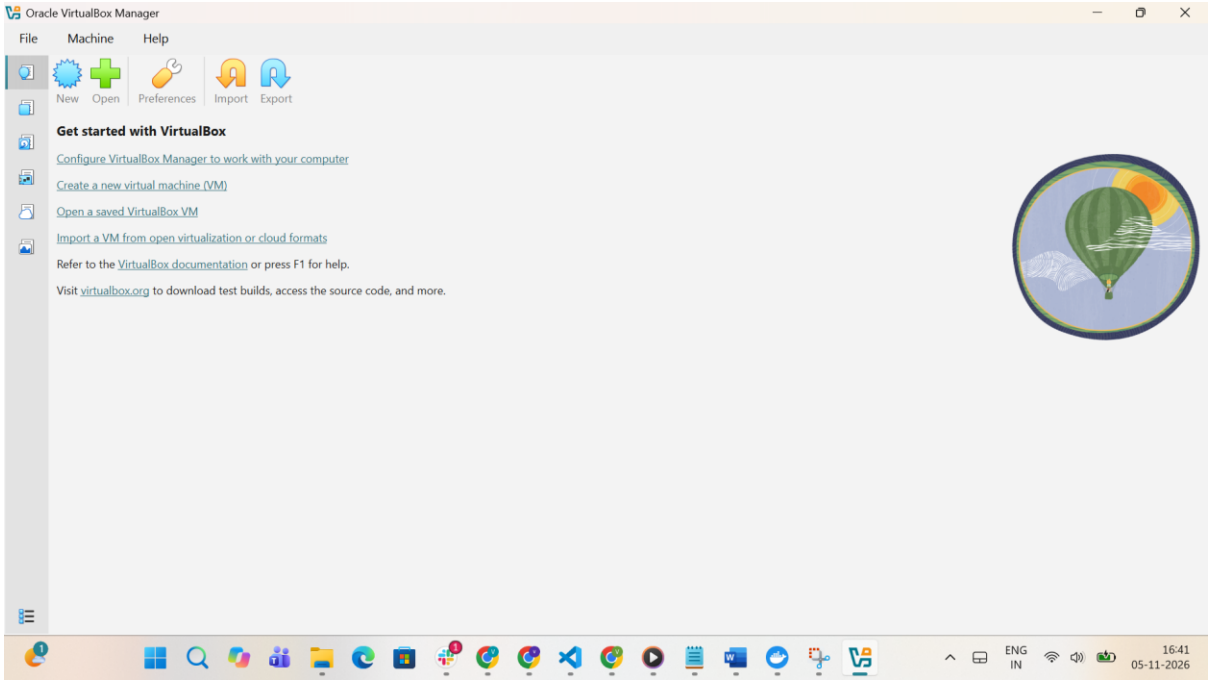
5) Set the minikube path to environmental varaible

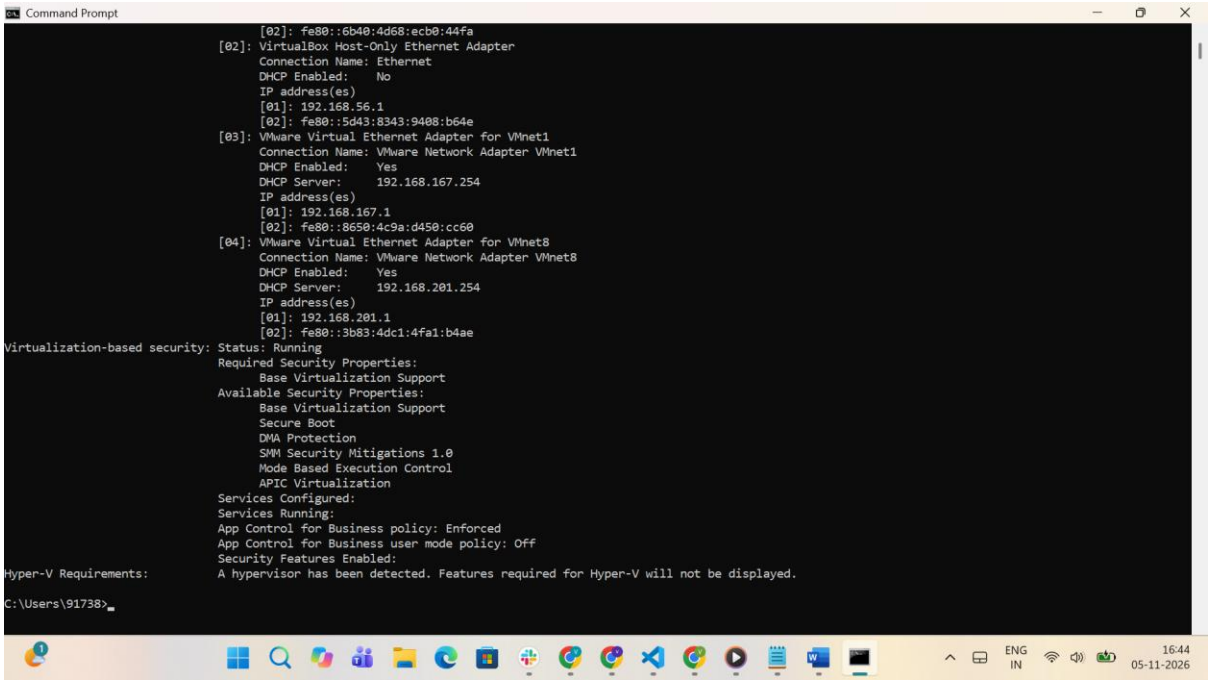
export PATH=$PATH:/c/minikube

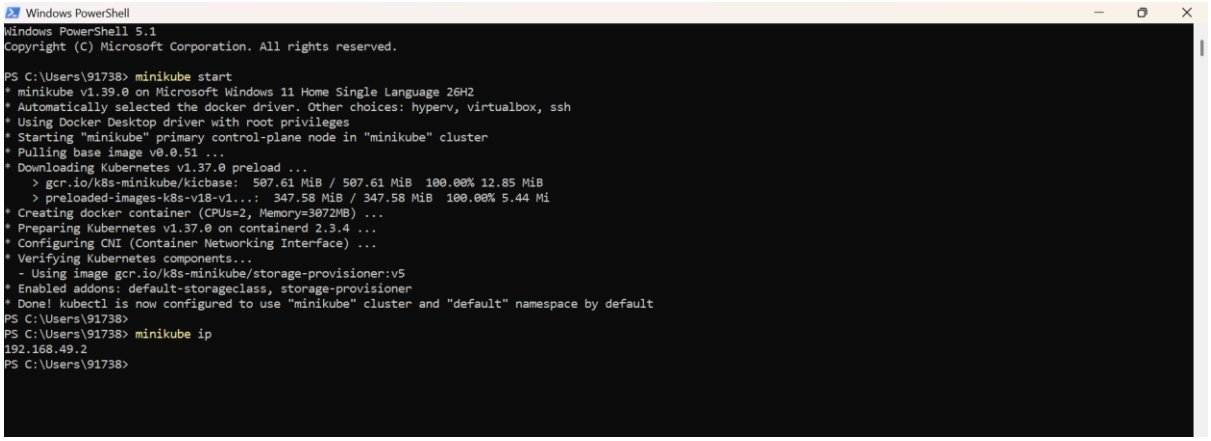
6) Restart powershell and start minikube

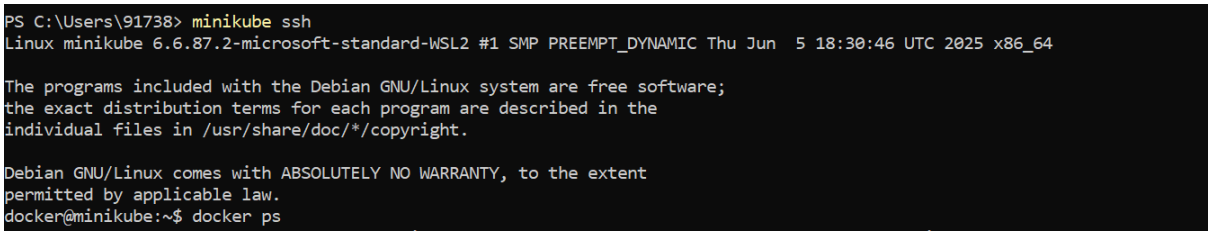
minikube start




Oracle VirtualBox Manager
File
Machine
Help
New
Open
Preferences
Import
Export
Get started with VirtualBox
Configure VirtualBox Manager to work with your computer
Create a new virtual machine (VM)
Open a saved VirtualBox VM
Import a VM from open virtualization or cloud formats
Refer to the VirtualBox documentation or press F1 for help.
Visit virtualbox.org to download test builds, access the source code, and more.
ENG
IN
16:41
05-11-2026


Command Prompt
[02]: fe80::6b40:4d68:ecb0:44fa
[02]: VirtualBox Host-Only Ethernet Adapter
Connection Name: Ethernet
DHCP Enabled: No
IP address(es)
[01]: 192.168.56.1
[02]: fe80::5d43:8343:9408:b64e
[03]: VMware Virtual Ethernet Adapter for VMnet1
Connection Name: VMware Network Adapter VMnet1
DHCP Enabled: Yes
DHCP Server: 192.168.167.254
IP address(es)
[01]: 192.168.167.1
[02]: fe80::8650:4c9a:d450:cc60
[04]: VMware Virtual Ethernet Adapter for VMnet8
Connection Name: VMware Network Adapter VMnet8
DHCP Enabled: Yes
DHCP Server: 192.168.201.254
IP address(es)
[01]: 192.168.201.1
[02]: fe80::3b83:4dc1:4fa1:b4ae
Virtualization-based security: Status: Running
Required Security Properties:
Base Virtualization Support
Available Security Properties:
Base Virtualization Support
Secure Boot
DMA Protection
SMM Security Mitigations 1.0
Mode Based Execution Control
APIC Virtualization
Services Configured:
Services Running:
App Control for Business policy: Enforced
App Control for Business user mode policy: Off
Security Features Enabled:
Hyper-V Requirements: A hypervisor has been detected. Features required for Hyper-V will not be displayed.
C:\Users\91738>
ENG
IN
16:44
05-11-2026


Windows PowerShell
Windows PowerShell 5.1
Copyright (C) Microsoft Corporation. All rights reserved.
PS C:\Users\91738> minikube start
* minikube v1.39.0 on Microsoft Windows 11 Home Single Language 26H2
* Automatically selected the docker driver. Other choices: hyperv, virtualbox, ssh
* Using Docker Desktop driver with root privileges
* Starting "minikube" primary control-plane node in "minikube" cluster
* Pulling base image v0.0.51 ...
* Downloading Kubernetes v1.37.0 preload ...
> gcr.io/k8s-minikube/kicbase: 507.61 MiB / 507.61 MiB 100.00% 12.85 MiB
> preloaded-images-k8s-v18-v1...: 347.58 MiB / 347.58 MiB 100.00% 5.44 Mi
* Creating docker container (CPUs=2, Memory=3072MB) ...
* Preparing Kubernetes v1.37.0 on containerd 2.3.4 ...
* Configuring CNI (Container Networking Interface) ...
* Verifying Kubernetes components...
- Using image gcr.io/k8s-minikube/storage-provisioner:v5
* Enabled addons: default-storageclass, storage-provisioner
* Done! kubectl is now configured to use "minikube" cluster and "default" namespace by default
PS C:\Users\91738>
PS C:\Users\91738> minikube ip
192.168.49.2
PS C:\Users\91738>



2.Setup k8s master and two worker nodes on ubuntu.

Implementations steps:

Step 1: Launch EC2 instances

1. Logged in to the AWS Management Console.
2. Opened the EC2 dashboard.
3. Launched three Ubuntu EC2 instances.
4. Named the instances:
5. master
6. worker-01
7. worker-02
8. Selected the required instance type (t2.medium as instructed by the trainer).
9. Configured the security group to allow SSH and Kubernetes cluster
10. communication.
11. Connected to all three instances using SSH.

Step 2: Disable Swap Memory

1. Executed the following commands on all three instances.
2. sudo swapoff -a
3. To disable swap permanently:
4. sudo sed -i '/ swap / s/^/#/' /etc/fstab
5. Verified using:
6. free -h

Step 3: Install Docker

1. Installed Docker on all three Ubuntu instances.
2. sudo apt update
3. sudo apt install -y docker.io
4. Started and enabled Docker:
5. sudo systemctl enable --now docker
6. Verified Docker:
7. sudo systemctl status docker

Step 4: Install Kubernetes Components

1. Configured the Kubernetes package repository on all three instances.
2. Installed the required packages:
3. sudo apt update

4. sudo apt install -y apt-transport-https ca-certificates curl gpg
5. Added the Kubernetes signing key and repository:
6. sudo mkdir -p -m 755 /etc/apt/keyrings
7. curl -fsSL https://pkgs.k8s.io/core:/stable:/v1.34/deb/Release.key | sudo gpg --dearmor -o /etc/apt/keyrings/kubernetes-apt-keyring.gpg
8. echo 'deb [signed-by=/etc/apt/keyrings/kubernetes-apt-keyring.gpg]
9. https://pkgs.k8s.io/core:/stable:/v1.34/deb/ /' | sudo tee
10. /etc/apt/sources.list.d/kubernetes.list
11. Updated the repository and installed Kubernetes components:
12. sudo apt update
13. sudo apt install -y kubelet kubeadm kubectl
14. Held the package versions to prevent automatic upgrades:
15. sudo apt-mark hold kubelet kubeadm kubectl
16. Enabled kubelet:
17. sudo systemctl enable --now kubelet

Step 5: Configure Containerd

1. Configured containerd as the Kubernetes container runtime on all three instances.
2. Generated the default containerd configuration:
3. sudo mkdir -p /etc/containerd
4. sudo containerd config default | sudo tee /etc/containerd/config.toml
5. Changed the SystemdCgroup setting from false to true:
6. sudo sed -i 's/SystemdCgroup = false/SystemdCgroup = true/'
7. /etc/containerd/config.toml
8. Restarted and enabled containerd:
9. sudo systemctl restart containerd
10. sudo systemctl enable containerd
11. Verified:
12. sudo systemctl status containerd

Step 6: Initialize the Kubernetes Master

1. Connected to the Master instance.
2. Initialized the Kubernetes cluster with the Flannel network CIDR:
3. sudo kubeadm init --pod-network-cidr=10.244.0.0/16
4. After successful initialization, Kubernetes generated a kubeadm join command for
5. adding the worker nodes.
6. Saved this command for later use.

Step 7: Configure kubectl on Master

1. Executed the following commands on the Master instance:
2. mkdir -p $HOME/.kube
3. sudo cp -i /etc/kubernetes/admin.conf $HOME/.kube/config
4. sudo chown $(id -u):$(id -g) $HOME/.kube/config
5. Verified the Master node:

6. kubectl get nodes

Step 8:Install Flannel Network Plugin

1. Installed Flannel as the Container Network Interface (CNI) plugin.
2. kubectl apply -f https://github.com/flannel-io/flannel/releases/latest/download/kube
3. flannel.yml
4. Initially, the Flannel pod encountered an issue because the br_netfilter kernel module
5. was missing.
6. Resolved it on the Master by executing:
7. sudo modprobe br_netfilter
8. Created the Kubernetes networking configuration:
9. sudo tee /etc/sysctl.d/k8s.conf <<EOF
10. net.bridge.bridge-nf-call-iptables = 1
11. net.bridge.bridge-nf-call-ip6tables = 1
12. net.ipv4.ip_forward = 1
13. EOF
14. Applied the settings:
15. sudo sysctl --system
16. Enabled the module permanently:
17. echo br_netfilter | sudo tee /etc/modules-load.d/k8s.conf
18. Deleted the failed Flannel pod so that Kubernetes could recreate it:
19. kubectl delete pod -n kube-flannel -l app=flannel
20. Verified the Flannel pod:
21. kubectl get pods -n kube-flannel
22. Confirmed that the Flannel pod was running successfully.

Step 9: Configure Networking on Worker Nodes

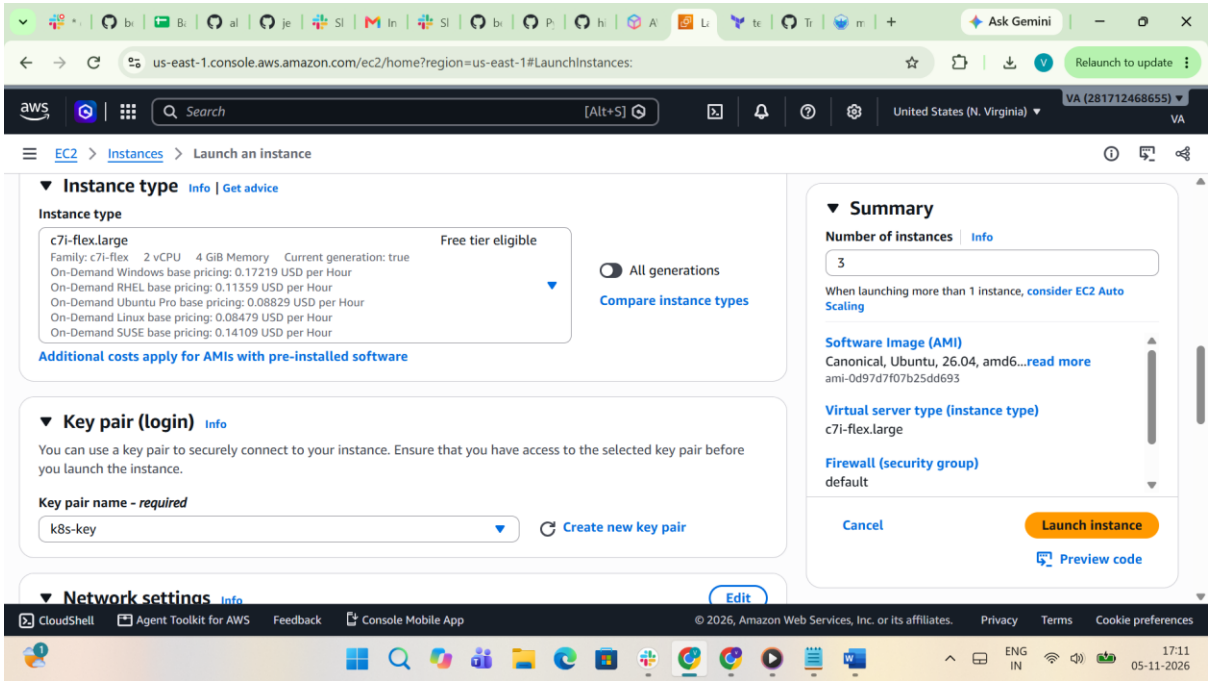
1. Performed the required networking configuration on the worker instances.
2. Loaded the br_netfilter module:
3. sudo modprobe br_netfilter
4. Created the networking configuration:
5. sudo tee /etc/sysctl.d/k8s.conf <<EOF
6. net.bridge.bridge-nf-call-iptables = 1
7. net.bridge.bridge-nf-call-ip6tables = 1
8. net.ipv4.ip_forward = 1
9. EOF
10. Applied the configuration:
11. sudo sysctl --system
12. Enabled the module permanently:
    a. echo br_netfilter | sudo tee /etc/modules-load.d/k8s.conf

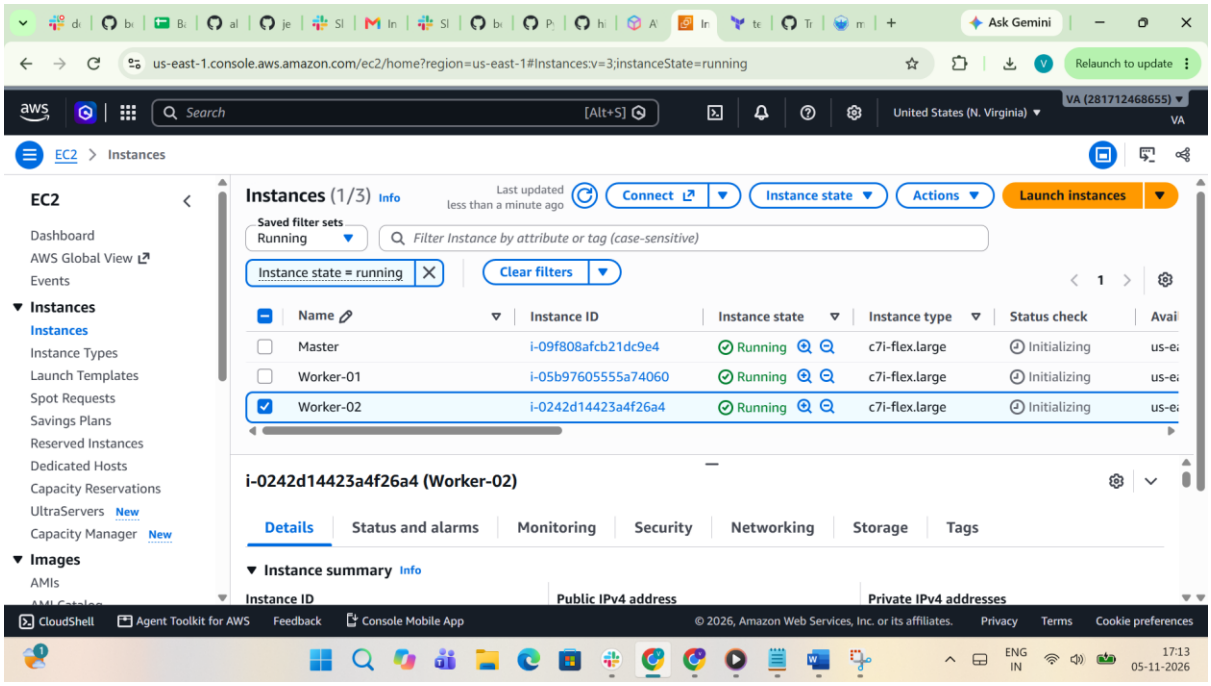
Step 10: Join Worker Nodes to the Master

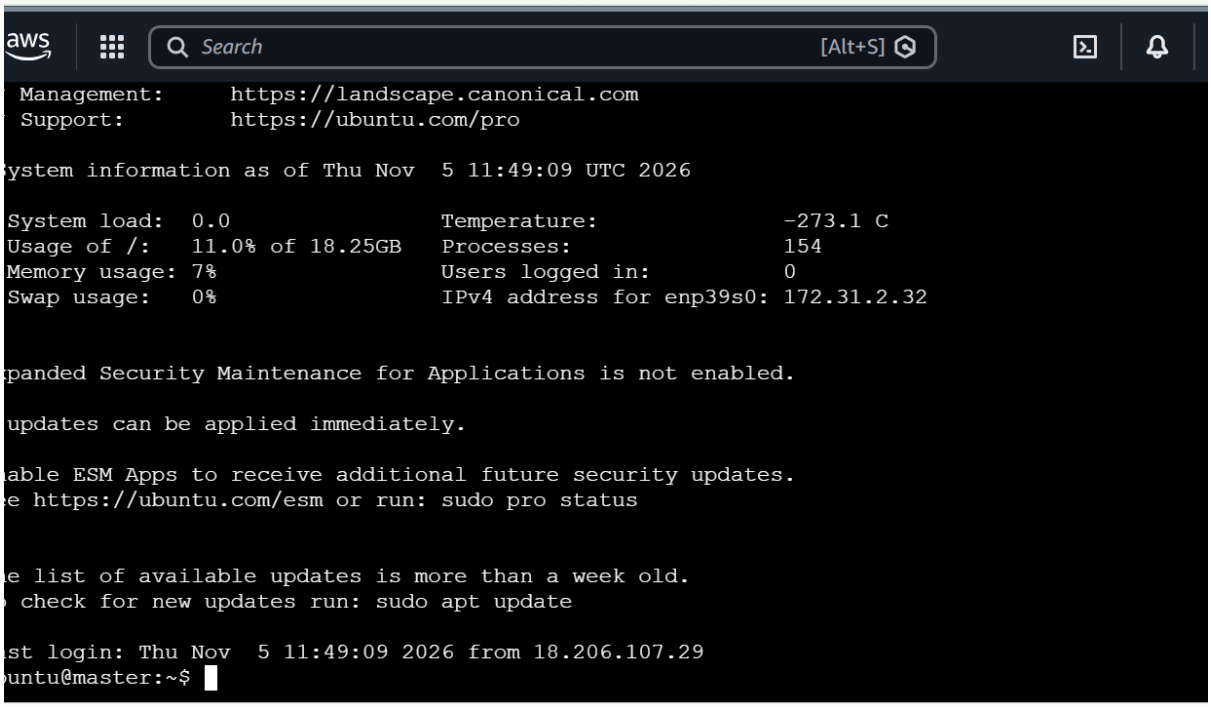
1. Executed the kubeadm join command generated during Master initialization on Worker
2. 1 and Worker 2.
3. Command used:
4. sudo kubeadm join 172.31.14.234:6443 --token <your-token> --discovery-token-ca
5. cert-hash sha256:<your-hash>
6. Both worker nodes successfully joined the cluster.
7. The terminal displayed:
8. This node has joined the cluster:
    a. Certificate signing request was sent to apiserver.
    b. The Kubelet was informed of the new secure connection details.

Step 11: Verify the Kubernetes Cluster

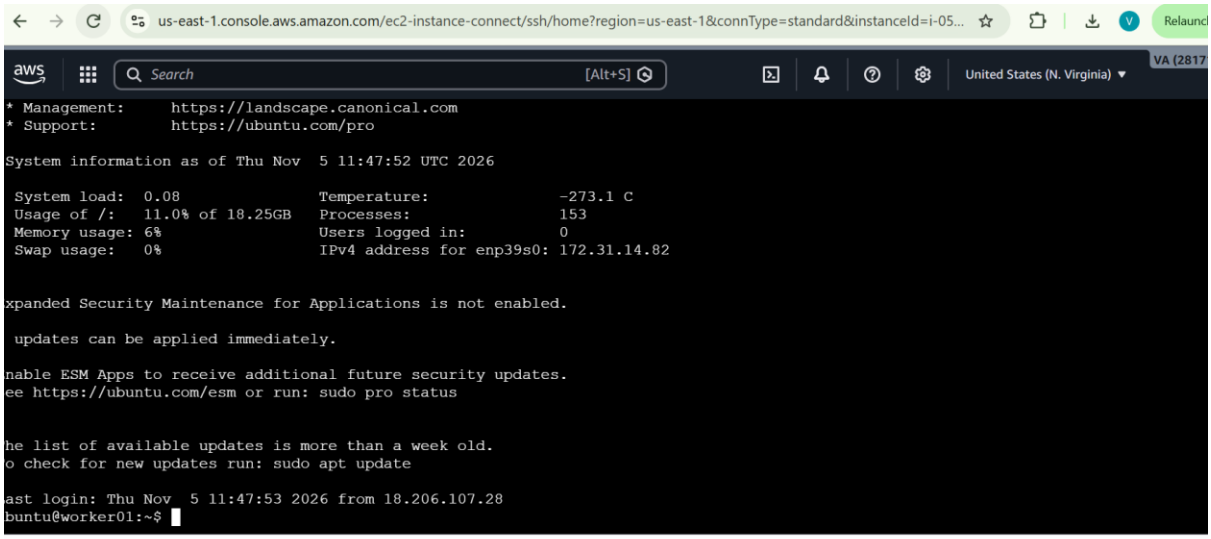
1. Returned to the Master instance and executed:
2. kubectl get nodes -o wide
3. Verified all three nodes were in Ready status.
4. Also verified all system pods:
5. kubectl get pods -A
6. Confirmed that Flannel, CoreDNS, etcd, API Server, Controller Manager, Scheduler and
7. kube-proxy were running.





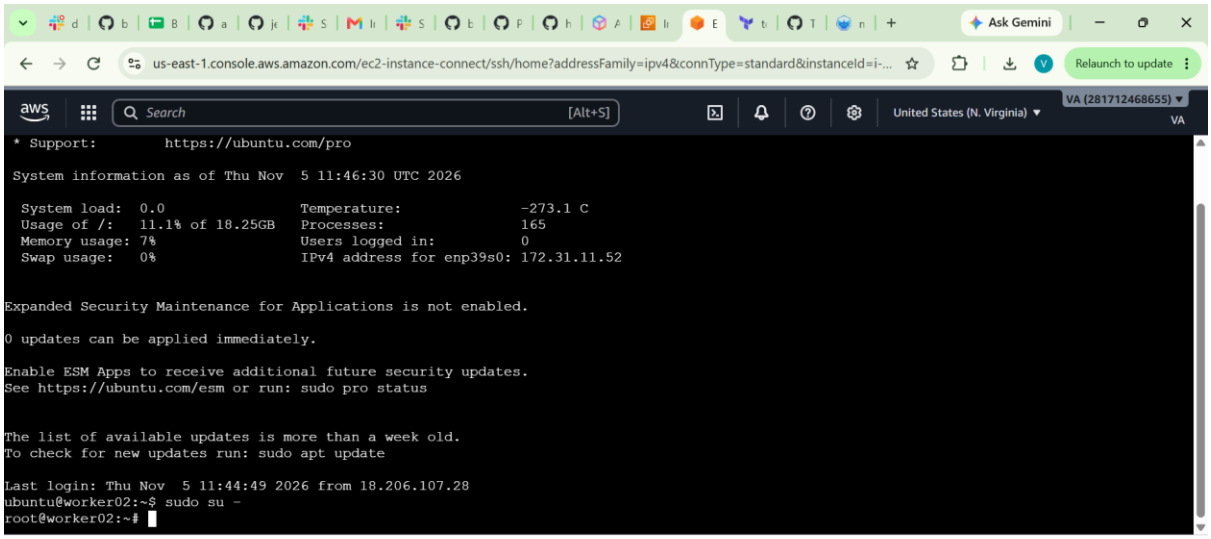


i-09f808afcb21dc9e4 (Master)



i-05b97605555a74060 (Worker-01)

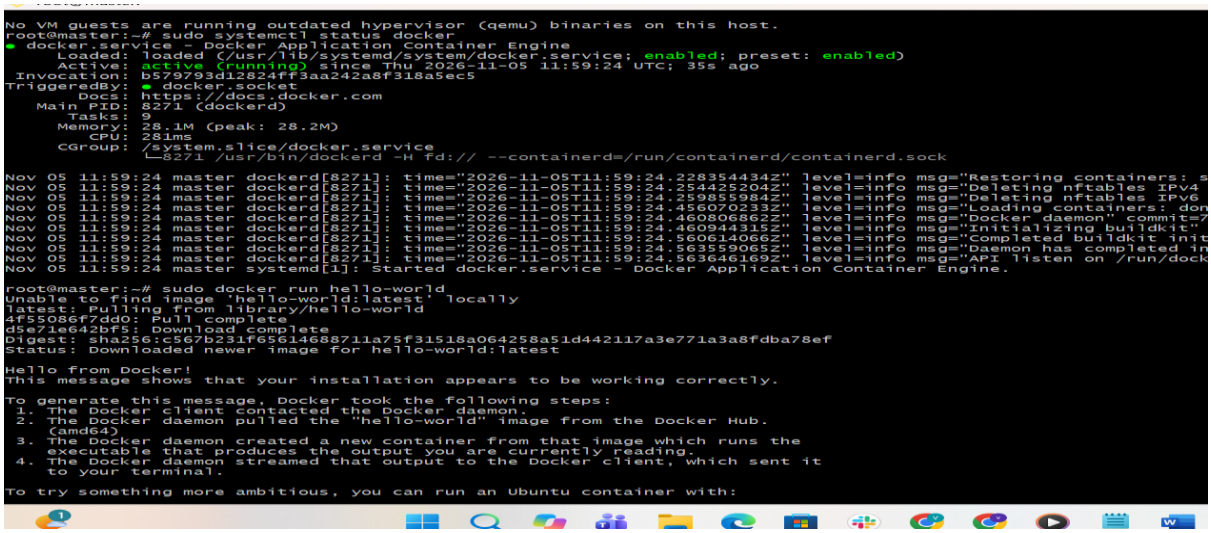
PublicIPs: 98.92.205.243 PrivateIPs: 172.31.14.82



i-0242d14423a4f26a4 (Worker-02)

PublicIPs: 100.53.61.152 PrivateIPs: 172.31.11.52





```
o containers need to be restarted.

o user sessions are running outdated binaries.

o VM guests are running outdated hypervisor (qemu) binaries on this host.
oot@worker01:~# sudo systemctl status docker
 docker.service - Docker Application Container Engine
     Loaded: loaded (/usr/lib/systemd/system/docker.service; enabled; preset: enabled)
     Active: active (running) since Thu 2026-11-05 11:59:35 UTC; 32s ago
Invocation: 3940c849b765434ea54286fcb382b265
TriggeredBy: ● docker.socket
       Docs: https://docs.docker.com
   Main PID: 8273 (dockerd)
      Tasks: 9
     Memory: 27.9M (peak: 28M)
        CPU: 310ms
     CGroup: /system.slice/docker.service
             └─8273 /usr/bin/dockerd -H fd:// --containerd=/run/containerd/containerd.sock

ov 05 11:59:34 worker01 dockerd[8273]: time="2026-11-05T11:59:34.686620971Z" level=info msg="Restoring containers: start."
ov 05 11:59:34 worker01 dockerd[8273]: time="2026-11-05T11:59:34.711851207Z" level=info msg="Deleting nftables IPv4 rules" error=
ov 05 11:59:34 worker01 dockerd[8273]: time="2026-11-05T11:59:34.716380939Z" level=info msg="Deleting nftables IPv6 rules" error=
ov 05 11:59:34 worker01 dockerd[8273]: time="2026-11-05T11:59:34.926946829Z" level=info msg="Loading containers: done."
ov 05 11:59:34 worker01 dockerd[8273]: time="2026-11-05T11:59:34.932233625Z" level=info msg="Docker daemon" commit=70efc0e contai
ov 05 11:59:34 worker01 dockerd[8273]: time="2026-11-05T11:59:34.932357143Z" level=info msg="Initializing buildkit"
ov 05 11:59:35 worker01 dockerd[8273]: time="2026-11-05T11:59:35.018889907Z" level=info msg="Completed buildkit initialization"
ov 05 11:59:35 worker01 dockerd[8273]: time="2026-11-05T11:59:35.023469904Z" level=info msg="Daemon has completed initialization"
ov 05 11:59:35 worker01 dockerd[8273]: time="2026-11-05T11:59:35.023537210Z" level=info msg="API listen on /run/docker.sock"
ov 05 11:59:35 worker01 systemd[1]: Started docker.service - Docker Application Container Engine.

oot@worker01:~# sudo docker run hello-world
nable to find image 'hello-world:latest' locally
atest: Pulling from library/hello-world
f55086f7dd0: Pull complete
5e71e642bf5: Download complete
igest: sha256:c567b231f65614688711a75f31518a064258a51d442117a3e771a3a8fdba78ef
tatus: Downloaded newer image for hello-world:latest

ello from Docker!
his message shows that your installation appears to be working correctly.

o generate this message, Docker took the following steps:
1. The Docker client contacted the Docker daemon.
2. The Docker daemon pulled the "hello-world" image from the Docker Hub.
   (amd64)
3. The Docker daemon created a new container from that image which runs the
   executable that produces the output you are currently reading.
```



```
root@worker02: ~

No containers need to be restarted.

No user sessions are running outdated binaries.

No VM guests are running outdated hypervisor (qemu) binaries on this host.
root@worker02:~# sudo systemctl status docker
● docker.service - Docker Application Container Engine
     Loaded: loaded (/usr/lib/systemd/system/docker.service; enabled; preset: enabled)
     Active: active (running) since Thu 2026-11-05 11:59:43 UTC; 30s ago
 Invocation: b6c1c8543dec4fcf9f3d1b01a6768cf4
TriggeredBy: ● docker.socket
       Docs: https://docs.docker.com
   Main PID: 8290 (dockerd)
      Tasks: 9
     Memory: 27.7M (peak: 27.7M)
        CPU: 250ms
     CGroup: /system.slice/docker.service
             └─8290 /usr/bin/dockerd -H fd:// --containerd=/run/containerd/containerd.sock

Nov 05 11:59:43 worker02 dockerd[8290]: time="2026-11-05T11:59:43.105638006Z" level=info msg="Restoring containers: start."
Nov 05 11:59:43 worker02 dockerd[8290]: time="2026-11-05T11:59:43.132409712Z" level=info msg="Deleting nftables IPv4 rules" error="running nft: /dev/stdin:1:17-30: Error:
Nov 05 11:59:43 worker02 dockerd[8290]: time="2026-11-05T11:59:43.135697852Z" level=info msg="Deleting nftables IPv6 rules" error="running nft: /dev/stdin:1:18-31: Error:
Nov 05 11:59:43 worker02 dockerd[8290]: time="2026-11-05T11:59:43.344625059Z" level=info msg="Loading containers: done."
Nov 05 11:59:43 worker02 dockerd[8290]: time="2026-11-05T11:59:43.351683886Z" level=info msg="Docker daemon" commit=70efc0e containerd-snapshotter=true storage-driver=ove
Nov 05 11:59:43 worker02 dockerd[8290]: time="2026-11-05T11:59:43.351775035Z" level=info msg="Initializing buildkit"
Nov 05 11:59:43 worker02 dockerd[8290]: time="2026-11-05T11:59:43.422856458Z" level=info msg="Completed buildkit initialization"
Nov 05 11:59:43 worker02 dockerd[8290]: time="2026-11-05T11:59:43.425844062Z" level=info msg="Daemon has completed initialization"
Nov 05 11:59:43 worker02 dockerd[8290]: time="2026-11-05T11:59:43.425918260Z" level=info msg="API listen on /run/docker.sock"
Nov 05 11:59:43 worker02 systemd[1]: Started docker.service - Docker Application Container Engine.

root@worker02:~# sudo docker run hello-world
Unable to find image 'hello-world:latest' locally
latest: Pulling from library/hello-world
4f55086f7dd0: Pull complete
d5e71e642bf5: Download complete
Digest: sha256:c567b231f65614688711a75f31518a064258a51d442117a3e771a3a8fdba78ef
Status: Downloaded newer image for hello-world:latest

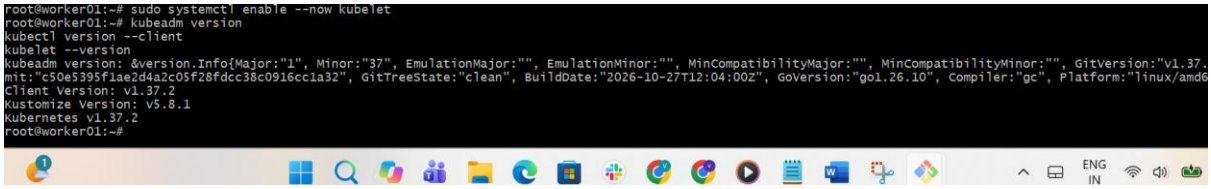
Hello from Docker!
This message shows that your installation appears to be working correctly.

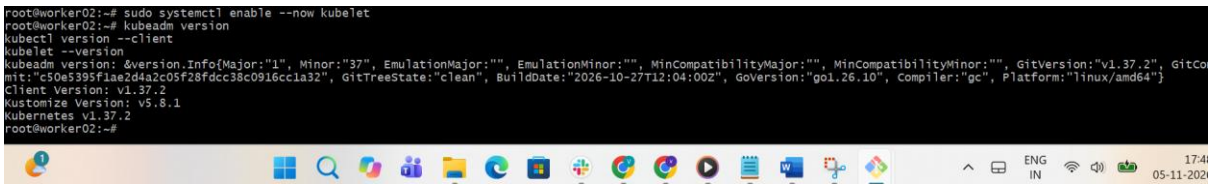
To generate this message, Docker took the following steps:
 1. The Docker client contacted the Docker daemon.
 2. The Docker daemon pulled the "hello-world" image from the Docker Hub.
    (amd64)
 3. The Docker daemon created a new container from that image which runs the
    executable that produces the output you are currently reading.
```

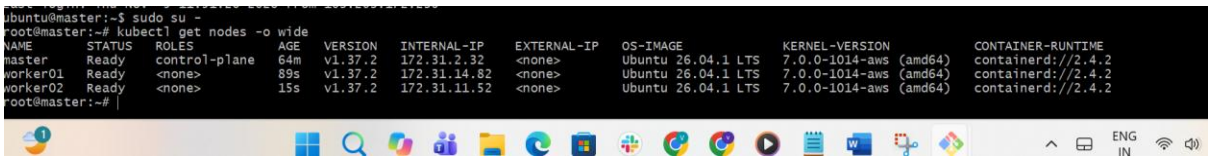


```
root@master:~# sudo systemctl enable --now kubelet
root@master:~# kubeadm version
kubectl version --client
kubelet --version
kubeadm version: &version.Info{Major:"1", Minor:"37", EmulationMajor:"", EmulationMinor:"", MinCompatibilityMajor:"", MinCompatibilityMinor:"", GitVersion:"v1.37.2",
mit:"c50e5395f1ae2d4a2c05f28fdcc38c0916cc1a32", GitTreeState:"clean", BuildDate:"2026-10-27T12:04:00Z", GoVersion:"go1.26.10", Compiler:"gc", Platform:"linux/amd64"}
Client Version: v1.37.2
Kustomize Version: v5.8.1
Kubernetes v1.37.2
root@master:~#
```









3.Run one nginx pod.

Implementations steps:

Step 1: Create an Nginx Pod

Connected to the Kubernetes Master and executed:
kubectl run nginx --image=nginx
This command created a Pod named nginx using the official Nginx Docker image.

Step 2: Verify the Pod

Executed:
kubectl get pods
Verified that the Nginx pod was running.
Expected output:
NAME READY STATUS RESTARTS AGE
nginx 1/1 Running 0

Step 3: Check Pod Details

Executed:
kubectl get pods -o wide
This displayed:
• Pod name
• Ready status
• Pod IP address
30s
• Worker node where the pod was scheduled

Step 4: Describe the Nginx Pod

Executed:
kubectl describe pod nginx
Verified the following details:
• Pod name: nginx
• Image: nginx

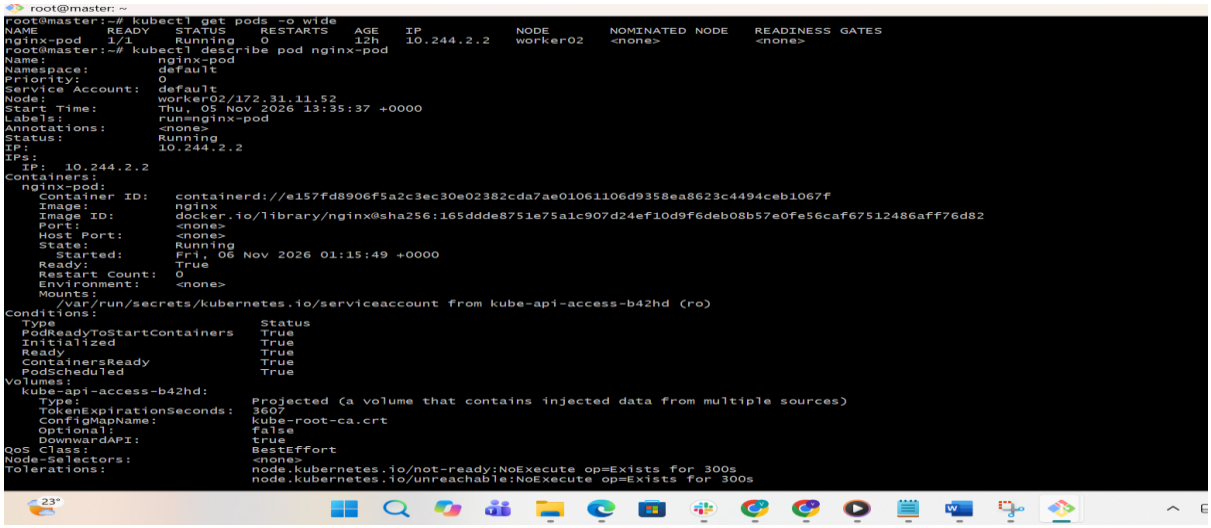
• Status: Running
• Ready: True
• Node: Worker node on which the pod was scheduled

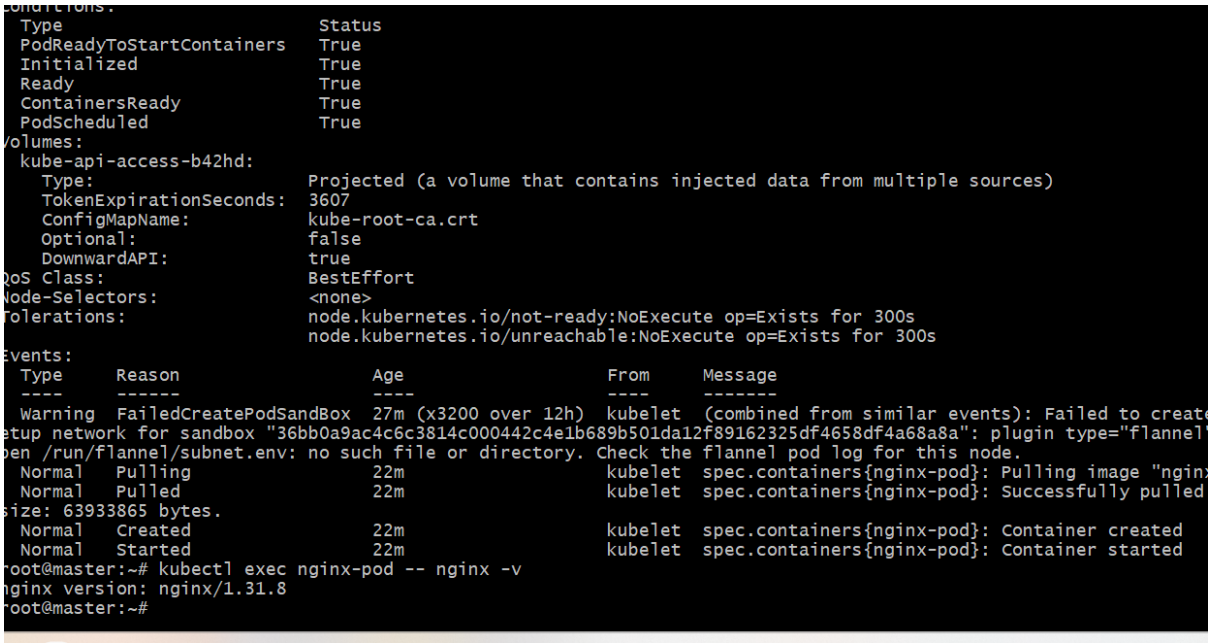
Step 5: Verify Nginx Version

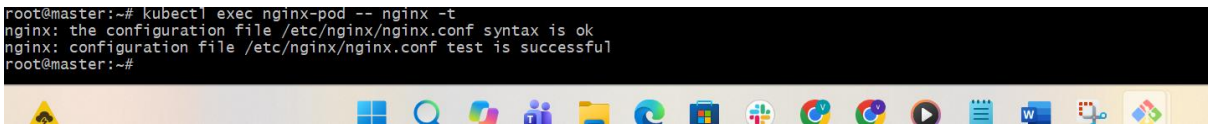
Executed:

kubectl exec nginx -- nginx -v

This verified that Nginx was installed and running inside the container.







4. Mugup Master and slave components on k8s.

Objective

To understand the architecture, components, functions, and working mechanism of Kubernetes Master (Control Plane) and Worker Nodes.

1. Introduction to Kubernetes

Kubernetes (K8s) is an open-source container orchestration platform used to automate the deployment, scaling, management, and monitoring of containerized applications.
It provides features such as:

- Automated application deployment
- Scaling of applications
- Load balancing
- Self-healing
- Rolling updates and rollbacks
- Resource management

2. Kubernetes Architecture

A Kubernetes cluster consists of two major parts:

1. Control Plane (Master Node): Responsible for managing and controlling the Kubernetes cluster.

2. Worker Nodes: Responsible for running application workloads in the form of Pods.
The Control Plane manages the cluster, while Worker Nodes execute the applications.

3. Master Node (Control Plane) Components

The Control Plane is responsible for maintaining the desired state of the cluster and managing the Worker Nodes.

3.1 API Server (kube-apiserver)

The API Server is the central communication component of Kubernetes. It acts as the entry point for all API requests and interactions with the cluster.
Functions:

- Accepts requests from kubectl and other components.
- Validates and processes API requests.
- Communicates with etcd to store and retrieve cluster data.
- Enables communication between Kubernetes components.

Example: When the user executes kubectl get nodes, the request is processed by the API Server, which returns the node information.

3.2 etcd

etcd is a distributed key-value database used by Kubernetes to store cluster data and configuration.
Functions:

- Stores cluster state and configuration.
- Maintains information about Nodes, Pods, Secrets, ConfigMaps, and other Kubernetes objects.
- Acts as the source of truth for the cluster state.
- Supports reliable storage of Kubernetes data.

Example: When a new Pod is created, its desired configuration is stored in etcd.

3.3 Scheduler (kube-scheduler)

The Scheduler is responsible for assigning newly created Pods to suitable Worker Nodes.

Functions:

• Identifies unscheduled Pods.
• Checks available CPU, memory, and other resource requirements.
• Considers scheduling constraints and policies.
• Selects the appropriate Worker Node for a Pod.

Example: When an Nginx Pod is created, the Scheduler selects a suitable Worker Node based on resource availability and scheduling requirements.

3.4 Controller Manager (kube-controller-manager)

The Controller Manager runs various controllers that continuously monitor the cluster and maintain its desired state.

Functions:

• Monitors the actual state of the cluster.
• Compares the actual state with the desired state.
• Takes corrective actions when differences are detected.
• Ensures that resources are maintained as configured.

Example: If a Deployment is configured with three replicas and one Pod fails, the controller works to create a replacement Pod to maintain the desired number of replicas.

3.5 Cloud Controller Manager (cloud-controller-manager)

The Cloud Controller Manager integrates Kubernetes with cloud platforms such as AWS, Azure, and Google Cloud.

Functions:

• Integrates Kubernetes with cloud infrastructure.
• Manages cloud-specific node information.
• Supports cloud load balancer provisioning.
• Manages cloud routes and related infrastructure.

Example: In AWS, a Kubernetes Service of type LoadBalancer can trigger the provisioning of an external AWS load balancer when the appropriate cloud integration is configured.

4. Worker Node Components

Worker Nodes are responsible for executing and maintaining application workloads assigned by the Control Plane.

4.1 Kubelet

Kubelet is an agent that runs on every Kubernetes node.

Functions:

• Communicates with the API Server.
• Receives Pod specifications assigned to its node.
• Ensures that containers inside the assigned Pods are running.
• Monitors Pod and node health.
• Reports node and Pod status to the Control Plane.

Example: When a Pod is scheduled on Worker Node 1, the Kubelet ensures that the

required containers are started and maintained.

4.2 Kube-proxy

Kube-proxy is a network component that runs on Kubernetes nodes and implements Service networking.

Functions:

• Maintains network rules on nodes.
• Enables communication between Services and backend Pods.
• Routes Service traffic to appropriate Pod endpoints.
• Supports network connectivity within the cluster.

Example: When a client accesses a Kubernetes Service, kube-proxy's networking rules help forward the traffic to an appropriate backend Pod.

4.3 Container Runtime

The Container Runtime is responsible for running and managing containers on Kubernetes nodes.

Functions:

• Pulls container images from registries.
• Creates and starts containers.
• Stops and manages container lifecycle.
• Provides container execution capabilities to Kubernetes through the Container Runtime Interface (CRI).

Examples:

• containerd
• CRI-O

In our practical implementation: We configured containerd as the container runtime on all three Ubuntu EC2 instances.

5. Difference Between Master and Worker Nodes

Master Node (Control Plane) Worker Node
Manages the Kubernetes cluster Runs application workloads
Controls cluster operations
Runs API Server
Runs etcd
Runs Scheduler
Runs Controller Manager
Executes assigned Pods
Runs Kubelet
Runs Container Runtime
Runs kube-proxy
Reports node and Pod status

6. Working Mechanism of Kubernetes

When a user deploys an application in Kubernetes, the following process takes place:

1. The user submits a request using kubectl.
2. The request reaches the API Server.
3. The API Server validates the request and stores the desired state in etcd.
4. The Scheduler identifies a suitable Worker Node for the Pod.

5. The scheduling decision is recorded through the API Server.
6. The Kubelet on the selected Worker Node receives the Pod specification.
7. Kubelet communicates with the Container Runtime to pull the image and start the container.
8. The Kubelet monitors the container and reports its status to the API Server.
9. The application Pod reaches the Running state.

7. Important Kubernetes Commands

Command	
kubectl get nodes	Displays cluster nodes
kubectl get pods -A	Displays Pods across all namespaces
kubectl get pods -n kube-system	Displays system Pods
kubectl get pods -n kube-flannel	Displays Flannel networking Pods
kubectl describe node master	Displays detailed Master node information
kubectl describe node worker-01	Displays detailed Worker node information
sudo systemctl status kubelet	Checks Kubelet service status
sudo systemctl status containerd	Checks container runtime status

8. Conclusion

Kubernetes follows a distributed architecture in which the Control Plane manages the cluster and Worker Nodes execute application workloads. The API Server, etcd, Scheduler, and Controller Manager are essential Control Plane components, while Kubelet, kube-proxy, and the Container Runtime are essential Worker Node components.

Understanding these components helps in managing, deploying, scaling, and troubleshooting applications in a Kubernetes cluster.